

Client instructions (Spanish) - ready to send

Gabriel, revise el deploy real del repositorio. La API no usa un subdominio separado: Caddy sirve todo desde el dominio principal. El frontend llama a la API por `/api/v1`, y Caddy redirige internamente `/api/*` al backend.

A. DNS para el sitio

En Cloudflare, por ahora configura:

Tipo	Nombre	Valor	Proxy
A	@	187.127.254.101	DNS only al inicio
A	www	187.127.254.101	opcional, DNS only al inicio
A	api	no hace falta	no crear por ahora

`api.mercadoahora.com.ar` no es necesario porque el backend esta publicado bajo:

```
https://mercadoahora.com.ar/api/v1
```

Para emitir el primer certificado SSL con Caddy, recomiendo dejar @ en DNS only temporalmente. Cuando `https://mercadoahora.com.ar` funcione, se puede volver a activar el proxy naranja de Cloudflare.

B. VPS / Caddy / SSL

En el VPS hay que editar:

```
/opt/mercado-ahora/.env.production
```

Valores principales:

```
APP_DOMAIN=mercadoahora.com.ar
APP_URL=https://mercadoahora.com.ar
ACME_EMAIL=no-reply@mercadoahora.com.ar
SESSION_SECURE_COOKIE=true
NEXT_PUBLIC_API_BASE_URL=/api/v1
NEXT_PUBLIC_STORAGE_URL=/storage
```

El problema actual es que Caddy esta tomando:

```
APP_DOMAIN=:80
```

Con eso, Caddy queda configurado solo para HTTP en puerto 80 y no para el dominio real con HTTPS. Cloudflare intenta conectar al origen y termina mostrando Error 522 porque el origen no esta respondiendo

correctamente para el dominio/SSL esperado.

Despues de editar el archivo, correr en el VPS:

```
cd /opt/mercado-ahora
docker compose --env-file .env.production -f docker-compose.prod.yml up -d --build
--remove-orphans
docker compose --env-file .env.production -f docker-compose.prod.yml ps
docker compose --env-file .env.production -f docker-compose.prod.yml logs -f caddy
```

Tambien verificar que esten abiertos los puertos:

```
80/tcp
443/tcp
```

En Cloudflare, el modo SSL recomendado es Full (strict), pero recien despues de que el certificado del origen este funcionando. Para el primer certificado recomiendo:

1. Poner DNS @ como DNS only.
2. Actualizar .env.production.
3. Reiniciar contenedores.
4. Probar <https://mercadoahora.com.ar>.
5. Cuando funcione, activar proxy naranja.
6. Dejar Cloudflare en Full (strict).

La alternativa es usar un Cloudflare Origin Certificate, pero con el Caddyfile actual habria que montar el certificado en el contenedor y ajustar Caddy. Para este deploy, es mas simple usar el certificado automatico de Caddy.

C. Resend email

En Resend hay que:

1. Agregar el dominio mercadoahora.com.ar.
2. Crear una API key.
3. Copiar los DNS que Resend muestre.
4. Configurarlos en Cloudflare.
5. Esperar verificacion del dominio.

Los registros exactos de Resend no se deben inventar: los genera el panel de Resend. El formato esperado sera algo asi:

Tipo	Nombre	Valor
TXT	@ o el host indicado por Resend	valor SPF indicado por Resend
CNAME/TXT	selector DKIM indicado por Resend	valor DKIM indicado por Resend

TXT	_dmarc	valor DMARC indicado por Resend
-----	--------	---------------------------------

Cuando Resend este verificado, en `/opt/mercado-ahora/.env.production`:

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.resend.com
MAIL_PORT=587
MAIL_SCHEME=smtp
MAIL_USERNAME=resend
MAIL_PASSWORD=REEMPLAZAR_CON_RESEND_API_KEY
MAIL_FROM_ADDRESS=no-reply@mercadoahora.com.ar
MAIL_FROM_NAME="Mercado Ahora"
```

Importante: el sistema ya usa el envio real de correos para recuperacion de contrasena y verificacion de email. El reset manual desde admin queda como herramienta de respaldo.

D. Checklist

1. Configurar DNS en Cloudflare: A @ -> 187.127.254.101.
2. Dejar @ temporalmente como DNS only.
3. Editar `/opt/mercado-ahora/.env.production`.
4. Reiniciar el deploy con Docker Compose.
5. Verificar que Caddy emita SSL.
6. Probar `https://mercadoahora.com.ar`.
7. Activar proxy naranja en Cloudflare.
8. Usar Cloudflare SSL en Full strict.
9. Agregar dominio en Resend.
10. Configurar DNS de Resend.
11. Agregar API key de Resend en `.env.production`.
12. Probar recuperacion de contrasena y verificacion de email desde una cuenta real.

E. Necesito de tu lado

- ? API key de Resend.
- ? Registros DNS exactos que Resend genere.
- ? Confirmacion del modo SSL actual en Cloudflare.
- ? Confirmacion de si quieres usar DNS only temporalmente o Origin Certificate.
- ? Confirmacion de si el registro @ esta proxied o DNS only.

Technical findings (English)

Repo findings

- ? `deploy/Caddyfile` uses `{ $APP_DOMAIN : : 80 }`.
- ? If `APP_DOMAIN` is empty, Caddy defaults to `:80`.
- ? Caddy routes:
 - ? `/api/*` to `backend:8000`
 - ? `/storage/*` to `backend:8000`
 - ? `/up` to `backend:8000`
 - ? everything else to `frontend:3000`
- ? No `api.mercadoahora.com.ar` is required.
- ? `docker-compose.prod.yml` exposes Caddy on ports `80:80` and `443:443`.
- ? `deploy/deploy.sh` creates `.env.production` only if missing, but later updates `APP_URL`, `APP_DOMAIN`, `ACME_EMAIL`, `SESSION_DOMAIN`, `SANCTUM_STATEFUL_DOMAINS`, and frontend public URLs when those env vars are provided.
- ? `deploy/deploy.sh` also updates `FRONTEND_URL` and `MAIL_*` values when GitHub Actions exports them, so Resend SMTP survives future deploys.
- ? If GitHub `APP_DOMAIN` secret is missing or empty, an existing `APP_DOMAIN=:80` can remain in place.
- ? `ACME_EMAIL` is stored/exported, but the current `deploy/Caddyfile` does not use it with a Caddy `global email` directive.

Required GitHub secrets

Currently documented/used:

```
VPS_HOST
VPS_USER
VPS_SSH_KEY
VPS_APP_DIR
APP_URL
APP_DOMAIN
ACME_EMAIL
MAIL_MAILER
MAIL_HOST
MAIL_PORT
MAIL_SCHEME
MAIL_USERNAME
MAIL_PASSWORD
MAIL_FROM_ADDRESS
MAIL_FROM_NAME
```

Workflow also references:

```
VPS_PASSWORD
```

That appears optional if SSH key auth works, but it is referenced in `.github/workflows/deploy.yml`.

Recommended values:

```
APP_URL=https://mercadoahora.com.ar
APP_DOMAIN=mercadoahora.com.ar
ACME_EMAIL=no-reply@mercadoahora.com.ar
```

Root cause of 522

Most likely origin configuration mismatch:

```
APP_DOMAIN=:80
```

means Caddy is not configured as a real HTTPS site for `mercadoahora.com.ar`. If Cloudflare connects to the origin expecting HTTPS/443, the origin is not correctly serving that domain/certificate.

Resend setup

Repo currently has:

```
MAIL_MAILER=log
```

Laravel mail config supports SMTP and has a `resend` transport entry, but `resend/resend.php` is not required in `composer.json`. So SMTP is the right path for now.

Verified against Resend official docs: Laravel SMTP uses `smtp.resend.com`, port 587, username `resend`, password as API key. In this repo, use `MAIL_SCHEME=smtp`; the local Symfony mailer build accepts `smtp/smtps` schemes and rejects `tls`.

Sources:

- ? [Resend Laravel SMTP docs](https://resend.com/docs/send-with-laravel-smtp)
- ? [Cloudflare Full strict docs](https://developers.cloudflare.com/ssl/origin-configuration/ssl-modes/full-strict/)
- ? [Cloudflare Flexible mode docs](https://developers.cloudflare.com/ssl/origin-configuration/ssl-modes/flexible/)

Developer action items

- ? Update GitHub secrets for `APP_URL`, `APP_DOMAIN`, `ACME_EMAIL`, and the `MAIL_*` values listed above.
- ? Manually edit VPS `.env.production` once it already contains `APP_DOMAIN=:80`.
- ? Consider updating `deploy/Caddyfile` to actually use `ACME_EMAIL`.
- ? Consider documenting the temporary DNS-only SSL issuance step.
- ? Consider adding explicit `www` support or redirect if client wants `www.mercadoahora.com.ar`.

Health checks after fix:

```
cd /opt/mercado-ahora
docker compose --env-file .env.production -f docker-compose.prod.yml ps
docker compose --env-file .env.production -f docker-compose.prod.yml logs --tail=100 caddy
curl -I -H "Host: mercadoahora.com.ar" http://127.0.0.1/up
curl -I https://mercadoahora.com.ar/up
curl -I https://mercadoahora.com.ar/api/v1/categories
```